

Assignment 4: Linked Lists and Monopoly

Name: _____ Date: October 2026

GitHub (submitted commit/tag): _____

Language: C++17 | GUI: SDL2 | File: DomMonopoly.cpp | Runs on JDoodle (Libraries = SDL2, GUI + Interactive on)

1. Overview

The program has two parts in one file. Part 1 is a hand-written circular singly linked list (no `std::list`) with add, search, remove and print, plus a set of PASS/FAIL tests. Part 2 uses that same list as a 40-space circular Monopoly board. When the program starts it runs the list tests, then a 12-turn computer-vs-computer simulation that prints every move and the final results, and then opens a playable SDL2 window (the GUI bonus).

2. Part 1: The Linked List

Each node is a `Space` struct holding the property's name, cost, owner and a `next` pointer (plus Monopoly data such as rent and houses). The `Board` class keeps a `head` pointer, a `tail` pointer and a `count`. The tail's `next` always points back to the head, which is what makes the board circular.

Operation	How it works
<code>add(info)</code>	Creates a node, links it after the tail, moves the tail, and points the tail back at the head.
<code>search(name)</code>	Starts at the head and follows next pointers until the name matches or it is back at the head.
<code>remove(name)</code>	Walks the list tracking the previous node, then relinks <code>prev->next</code> past the removed node. Handles removing the head, the tail, and the last remaining node.
<code>print()</code>	Visits each node once (do/while until back at the head) and prints its index, name, cost and owner.
<code>~Board()</code>	Deletes exactly <i>count</i> nodes, so the circular link never causes an infinite loop or double free.

The tests check adding three items, finding and missing a search, removing the middle and the head, that the list stays circular after removals, that a single node points to itself, and that removing a missing item returns false. All eight pass (see the console output in section 5).

3. Part 2: Monopoly Game

Requirement	Where it is met
Circular board built from my linked list, at least 10 properties	<code>Game()</code> adds 40 spaces to a <code>Board</code> : 28 buyable properties (22 streets, 4 railroads, 2 utilities) plus GO, Jail, taxes, Chance, Community Chest, Free Parking and Go To Jail.
Each property stores name, cost, owner, next	Fields of the <code>Space</code> node. owner is -1 for the bank, 0 for P1, 1 for P2.
Players move and wrap around	<code>moveForward()</code> follows next once per space. After the last space it lands back on GO and the player collects \$200.

Requirement	Where it is met
Buy unowned, cannot buy owned	tryBuy() refuses a space that is owned, not for sale, or unaffordable. The simulation runs explicit checks that print PASS.
Simulate at least 10 turns and print results	simulateTurns(12) prints each roll, landing, purchase and payment, standings after every turn, and the final board.
GUI (bonus)	SDL2 window: board drawn by walking the list, dice, tokens, side panel with buttons, info card for any clicked space, and a game log.

Extra Monopoly rules I added

- **Money:** \$1,500 to start, \$200 for passing GO, income and luxury tax, Chance and Community Chest cards, and an auction when a player turns down a property.
- **Rent:** street rent doubles with a full color set and rises with houses; railroads charge \$25/\$50/\$100/\$200 by how many are owned; utilities charge 4x or 10x the dice; mortgaged properties charge nothing.
- **Jail:** reached by the Go To Jail space, a card, or three doubles in a row (no \$200). Get out by rolling doubles, paying \$50, or using a Get Out of Jail Free card; after three misses the player pays \$50 and moves.
- **Houses and hotels:** only on a complete, unmortgaged color set, built evenly; the fifth level is a hotel. Buildings sell back at half price, and properties can be mortgaged for half their cost (unmortgage = mortgage + 10%).
- **Win and loss:** a player who can't pay must sell or mortgage. If that still isn't enough they go bankrupt, their assets go to the player they owed, and the other player wins.

4. Time Complexity

n = number of spaces on the board (40), k = number of spaces moved in one roll.

Operation	Cost	Why
Adding	$O(1)$	The tail pointer lets a node be linked at the end without walking the list.
Searching	$O(n)$	A singly linked list has no index, so it may have to check every node before finding the name (or confirming it is missing).
Removing	$O(n)$	Finding the node and the node before it is a walk; the relinking itself is $O(1)$.
Moving	$O(k)$	One pointer hop per space. A roll is at most 12, so each move is effectively constant time.
Traversing	$O(n)$	Printing, drawing, or checking who owns a color set visits each node exactly once.

Trade-off: an array would make jumping to a space $O(1)$, but the linked list matches the assignment and makes moving around the circle natural, since following next from the last space returns to GO with no special case.

5. Output Screenshots

Console output

Linked list tests, then the 12-turn simulation. The full starting board print (40 lines, all owned by the bank) is shortened here; the final board is shown in full.

```
=== PART 1: LINKED LIST TESTS ===
[0] Alpha $100 owner: bank
[1] Beta $200 owner: bank
[2] Gamma $300 owner: bank
PASS: add three items
PASS: search finds Beta
PASS: search misses Zeta
PASS: remove middle item
PASS: still circular after removing middle
PASS: remove head item
PASS: single node points to itself
PASS: removing a missing item returns false
[0] Gamma $300 owner: bank

=== PART 2: MONOPOLY SIMULATION (12 TURNS) ===
Board (circular linked list):
[0] GO
[1] Cedar Ave $60 owner: bank
[2] Community Chest
[3] Birch St $60 owner: bank
[4] Income Tax
[5] North Rail $200 owner: bank
...
[39] Skyline Pl $400 owner: bank

--- TURN 1: P1 ---
P1 ROLLED 3 + 3 DOUBLES
P1 LANDED ON Harbor Rd
P1 BOUGHT Harbor Rd FOR $100
P1 ROLLED 3 + 6
P1 LANDED ON East Rail
P1 BOUGHT East Rail FOR $200
[after turn 1] P1 $1200, 2 properties, net worth $1500
                P2 $1500, 0 properties, net worth $1500

--- TURN 2: P2 ---
P2 ROLLED 6 + 2
P2 LANDED ON Pier Way
P2 BOUGHT Pier Way FOR $100
[after turn 2] P1 $1200, 2 properties, net worth $1500
                P2 $1400, 1 properties, net worth $1500

--- TURN 3: P1 ---
P1 ROLLED 2 + 3
P1 LANDED ON Free Parking
[after turn 3] P1 $1200, 2 properties, net worth $1500
                P2 $1400, 1 properties, net worth $1500

--- TURN 4: P2 ---
P2 ROLLED 3 + 1
P2 LANDED ON Power Co
P2 BOUGHT Power Co FOR $150
[after turn 4] P1 $1200, 2 properties, net worth $1500
                P2 $1250, 2 properties, net worth $1500

--- TURN 5: P1 ---
P1 ROLLED 4 + 4 DOUBLES
P1 LANDED ON Water Co
P1 BOUGHT Water Co FOR $150
P1 ROLLED 1 + 5
P1 LANDED ON Jade Blvd
P1 BOUGHT Jade Blvd FOR $320
[after turn 5] P1 $730, 4 properties, net worth $1500
                P2 $1250, 2 properties, net worth $1500

--- TURN 6: P2 ---
P2 ROLLED 1 + 6
P2 LANDED ON Amber Way
P2 BOUGHT Amber Way FOR $200
[after turn 6] P1 $730, 4 properties, net worth $1500
                P2 $1050, 3 properties, net worth $1500

--- TURN 7: P1 ---
P1 ROLLED 1 + 5
```

```

P1 PASSED GO +$200
P1 LANDED ON GO
  [after turn 7] P1 $930, 4 properties, net worth $1700
                  P2 $1050, 3 properties, net worth $1500
--- TURN 8: P2 ---
P2 ROLLED 4 + 4 DOUBLES
P2 LANDED ON Lemon Ave
P2 BOUGHT Lemon Ave FOR $260
P2 ROLLED 6 + 6 DOUBLES
P2 LANDED ON Skyline Pl
P2 BOUGHT Skyline Pl FOR $400
P2 ROLLED 6 + 1
P2 PASSED GO +$200
P2 LANDED ON Harbor Rd
P2 PAID P1 $6 RENT
  [after turn 8] P1 $936, 4 properties, net worth $1706
                  P2 $584, 5 properties, net worth $1694
--- TURN 9: P1 ---
P1 ROLLED 3 + 5
P1 LANDED ON Pier Way
P1 PAID P2 $6 RENT
  [after turn 9] P1 $930, 4 properties, net worth $1700
                  P2 $590, 5 properties, net worth $1700
--- TURN 10: P2 ---
P2 ROLLED 6 + 1
P2 LANDED ON Ridge Rd
P2 BOUGHT Ridge Rd FOR $140
  [after turn 10] P1 $930, 4 properties, net worth $1700
                  P2 $450, 6 properties, net worth $1700
--- TURN 11: P1 ---
P1 ROLLED 1 + 1 DOUBLES
P1 LANDED ON Jail
JUST VISITING
P1 ROLLED 2 + 3
P1 LANDED ON East Rail
  [after turn 11] P1 $930, 4 properties, net worth $1700
                  P2 $450, 6 properties, net worth $1700
--- TURN 12: P2 ---
P2 ROLLED 2 + 2 DOUBLES
P2 LANDED ON Community Chest
CHEST: GO TO JAIL
P2 GOES TO JAIL
  [after turn 12] P1 $930, 4 properties, net worth $1700
                  P2 $450, 6 properties, net worth $1700 (in jail)

--- Buy rule check ---
P2 CANNOT BUY Harbor Rd: OWNED BY P1
  PASS: player cannot buy a property someone already owns
P1 CANNOT BUY GO: NOT FOR SALE
  PASS: player cannot buy GO (not a property)

--- Results after 12 turns ---
P1: $930 cash, 4 properties, net worth $1700, on East Rail
P2: $450 cash, 6 properties, net worth $1700, in jail, on Jail
Leader: P1
Final board:
[0] GO
[1] Cedar Ave $60 owner: bank
[2] Community Chest
[3] Birch St $60 owner: bank
[4] Income Tax
[5] North Rail $200 owner: bank
[6] Harbor Rd $100 owner: P1
[7] Chance
[8] Pier Way $100 owner: P2
[9] Dock St $120 owner: bank
[10] Jail
[11] Canyon Dr $140 owner: bank
[12] Power Co $150 owner: P2
[13] Ridge Rd $140 owner: P2
[14] Summit Ave $160 owner: bank
[15] East Rail $200 owner: P1
[16] Dusk Blvd $180 owner: bank
[17] Community Chest
[18] Ember St $180 owner: bank
[19] Amber Way $200 owner: P2
[20] Free Parking

```

[21] Ruby Rd \$220 owner: bank
[22] Chance
[23] Garnet Ave \$220 owner: bank
[24] Cherry Ln \$240 owner: bank
[25] South Rail \$200 owner: bank
[26] Gold St \$260 owner: bank
[27] Lemon Ave \$260 owner: P2
[28] Water Co \$150 owner: P1
[29] Sunny Way \$280 owner: bank
[30] Go To Jail
[31] Fern Ave \$300 owner: bank
[32] Ivy Ln \$300 owner: bank
[33] Community Chest
[34] Jade Blvd \$320 owner: P1
[35] West Rail \$200 owner: bank
[36] Chance
[37] Ocean Dr \$350 owner: bank
[38] Luxury Tax
[39] Skyline Pl \$400 owner: P2

GUI



Figure 1. Start of a new game. The 40 spaces are drawn by walking the linked list; color bars show property groups, and both tokens start on GO.



Figure 2. Jail (turn 9). P1 rolled doubles three times and was sent to jail; P2 drew a Go To Jail card. Jailed tokens show bars, and the options are Roll Doubles or Pay \$50.

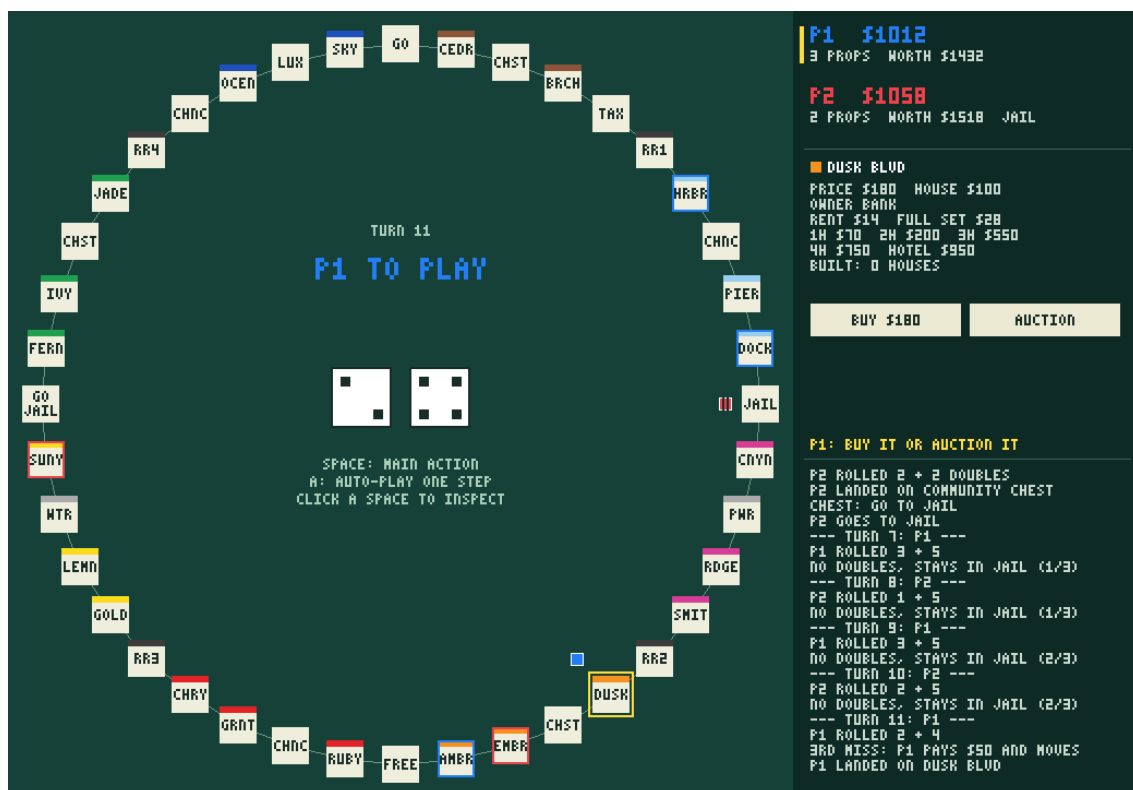


Figure 3. Buying (turn 11). After a third missed roll P1 pays \$50, leaves jail and lands on unowned Dusk Blvd, so the Buy / Auction choice appears. Owned spaces have a border in the owner's color.



Figure 4. Building. P1 owns the full red set and has built evenly (green squares are houses). The info card shows Ruby Rd's rent with 2 houses is \$250.



Figure 5. Win condition. P2 lands on Ruby Rd with a hotel, owes \$1,050 rent, cannot raise it and goes bankrupt. P1 takes P2's properties and wins.

6. Source Code (DomMonopoly.cpp)

```
1 // JDoodle: Libraries = SDL2; enable GUI and Interactive.
2 // Assignment 4: Linked Lists and Monopoly (GUI version)
3 //
4 // Controls:
5 //   Click the buttons on the right (or press SPACE for the main action).
6 //   Click the dice to roll. Click any board space to inspect it, then use
7 //   BUILD / SELL / MORTGAGE buttons on your own properties.
8 //   Press A to let the computer play one step for whoever is up (hold A to fast-forward).
9 //
10 // Before the window opens, the program runs the linked list tests and a 12-turn
11 // computer-vs-computer simulation and prints the results to the console.
12 //
13 // Time complexity (n = spaces on the board, k = spaces moved):
14 //   add      0(1)  tail pointer, so no walk to the end
15 //   search   0(n)  walk node by node until the name matches
16 //   remove   0(n)  walk to find the node and the node before it
17 //   move     0(k)  one pointer hop per space (k <= 12 per roll, so effectively 0(1))
18 //   traverse 0(n)  printing or drawing visits every node once
19 #include <SDL2/SDL.h>
20 #include <cctype>
21 #include <cmath>
22 #include <cstdlib>
23 #include <ctime>
24 #include <deque>
25 #include <iostream>
26 #include <string>
27 #include <vector>
28 using namespace std;
29
30 const double PI = 3.14159265358979;
31
32 // =====
33 // PART 1: CIRCULAR SINGLY LINKED LIST (no std::list)
34 // =====
35 enum SpaceType { GO_SPACE, STREET, RAILROAD, UTILITY, TAX, CHANCE, CHEST, JAIL, FREE_PARKING, GO_TO_JAIL };
36
37 struct SpaceInfo {
38     string name, label; // full name, short label drawn on the tile
39     SpaceType type;
40     int cost; // purchase price (or tax amount)
41     int group; // color group 0-7, 8 = railroads, 9 = utilities, -1 = none
42     int houseCost;
43     int rent[6]; // base rent, 1-4 houses, hotel
44 };
45
46 struct Space {
47     string name, label;
48     SpaceType type;
49     int cost;
50     int owner; // -1 = bank, 0 = P1, 1 = P2
51     Space* next;
52     int group, houseCost, rent[6];
53     int houses; // 0-4 houses, 5 = hotel
54     bool mortgaged;
55     int index;
56 };
57
58 class Board {
59 public:
60     Board() = default;
61     Board(const Board&) = delete;
62     Board& operator=(const Board&) = delete;
63     ~Board() { clear(); }
64
65     // Add to the end. 0(1) because we keep a tail pointer.
66     void add(const SpaceInfo& info) {
67         Space* s = new Space;
68         s->name = info.name;
69         s->label = info.label;
70         s->type = info.type;
71         s->cost = info.cost;
72         s->owner = -1;
73         s->group = info.group;
74         s->houseCost = info.houseCost;
75         for (int i = 0; i < 6; i++) s->rent[i] = info.rent[i];
76         s->houses = 0;
77         s->mortgaged = false;
78         s->index = count;
79         if (!head) head = tail = s;
80         else {
81             tail->next = s;
```

```

82         tail = s;
83     }
84     tail->next = head; // keep it circular
85     count++;
86 }
87
88 // Find by name. O(n).
89 Space* search(const string& name) const {
90     if (!head) return nullptr;
91     Space* cur = head;
92     do {
93         if (cur->name == name) return cur;
94         cur = cur->next;
95     } while (cur != head);
96     return nullptr;
97 }
98
99 // Remove by name. O(n): we must walk to find the node and its predecessor.
100 bool remove(const string& name) {
101     if (!head) return false;
102     Space* prev = tail;
103     Space* cur = head;
104     for (int i = 0; i < count; i++) {
105         if (cur->name == name) {
106             if (count == 1) head = tail = nullptr;
107             else {
108                 prev->next = cur->next;
109                 if (cur == head) head = cur->next;
110                 if (cur == tail) tail = prev;
111             }
112             delete cur;
113             count--;
114             renumber();
115             return true;
116         }
117         prev = cur;
118         cur = cur->next;
119     }
120     return false;
121 }
122
123 // Print every node once. O(n).
124 void print() const {
125     if (!head) {
126         cout << " (empty list)\n";
127         return;
128     }
129     const Space* cur = head;
130     do {
131         cout << " [" << cur->index << "] " << cur->name;
132         if (cur->type == STREET || cur->type == RAILROAD || cur->type == UTILITY) {
133             cout << " $" << cur->cost << " owner: "
134                 << (cur->owner < 0 ? string("bank") : "P" + to_string(cur->owner + 1));
135             if (cur->houses == 5) cout << " hotel";
136             else if (cur->houses > 0) cout << " houses: " << cur->houses;
137             if (cur->mortgaged) cout << " (mortgaged)";
138         }
139         cout << '\n';
140         cur = cur->next;
141     } while (cur != head);
142 }
143
144 // Walk to a position. O(n).
145 Space* at(int index) const {
146     Space* cur = head;
147     for (int i = 0; i < index; i++) cur = cur->next;
148     return cur;
149 }
150
151 Space* first() const { return head; }
152 int size() const { return count; }
153
154 void clear() {
155     for (int i = 0; i < count; i++) {
156         Space* next = head->next;
157         delete head;
158         head = next;
159     }
160     head = tail = nullptr;
161     count = 0;
162 }
163
164 private:
165     void renumber() {

```

```

166     Space* cur = head;
167     for (int i = 0; i < count; i++, cur = cur->next) cur->index = i;
168 }
169 Space* head = nullptr;
170 Space* tail = nullptr;
171 int count = 0;
172 };
173
174 void check(bool ok, const string& what) { cout << (ok ? " PASS: " : " FAIL: ") << what << '\n'; }
175
176 void testLinkedList() {
177     cout << "=== PART 1: LINKED LIST TESTS ===\n";
178     Board t;
179     t.add({"Alpha", "A", STREET, 100, 0, 50, {1, 2, 3, 4, 5, 6}});
180     t.add({"Beta", "B", STREET, 200, 0, 50, {1, 2, 3, 4, 5, 6}});
181     t.add({"Gamma", "G", STREET, 300, 0, 50, {1, 2, 3, 4, 5, 6}});
182     t.print();
183     check(t.size() == 3, "add three items");
184     check(t.search("Beta") && t.search("Beta")->cost == 200, "search finds Beta");
185     check(t.search("Zeta") == nullptr, "search misses Zeta");
186     check(t.remove("Beta") && !t.search("Beta") && t.size() == 2, "remove middle item");
187     check(t.first()->next->next == t.first(), "still circular after removing middle");
188     check(t.remove("Alpha") && t.first()->name == "Gamma", "remove head item");
189     check(t.first()->next == t.first(), "single node points to itself");
190     check(!t.remove("Zeta"), "removing a missing item returns false");
191     t.print();
192 }
193
194 // =====
195 // PART 2: MONOPOLY
196 // =====
197 const SpaceInfo BOARD_DATA[] = {
198     {"GO", "GO", GO_SPACE, 0, -1, 0, {}},
199     {"Cedar Ave", "CEDR", STREET, 60, 0, 50, {2, 10, 30, 90, 160, 250}},
200     {"Community Chest", "CHST", CHEST, 0, -1, 0, {}},
201     {"Birch St", "BRCH", STREET, 60, 0, 50, {4, 20, 60, 180, 320, 450}},
202     {"Income Tax", "TAX", TAX, 200, -1, 0, {}},
203     {"North Rail", "RR1", RAILROAD, 200, 8, 0, {}},
204     {"Harbor Rd", "HRBR", STREET, 100, 1, 50, {6, 30, 90, 270, 400, 550}},
205     {"Chance", "CHNC", CHANCE, 0, -1, 0, {}},
206     {"Pier Way", "PIER", STREET, 100, 1, 50, {6, 30, 90, 270, 400, 550}},
207     {"Dock St", "DOCK", STREET, 120, 1, 50, {8, 40, 100, 300, 450, 600}},
208     {"Jail", "JAIL", JAIL, 0, -1, 0, {}},
209     {"Canyon Dr", "CNYN", STREET, 140, 2, 100, {10, 50, 150, 450, 625, 750}},
210     {"Power Co", "PWR", UTILITY, 150, 9, 0, {}},
211     {"Ridge Rd", "RDGE", STREET, 140, 2, 100, {10, 50, 150, 450, 625, 750}},
212     {"Summit Ave", "SMIT", STREET, 160, 2, 100, {12, 60, 180, 500, 700, 900}},
213     {"East Rail", "RR2", RAILROAD, 200, 8, 0, {}},
214     {"Dusk Blvd", "DUSK", STREET, 180, 3, 100, {14, 70, 200, 550, 750, 950}},
215     {"Community Chest", "CHST", CHEST, 0, -1, 0, {}},
216     {"Ember St", "EMBR", STREET, 180, 3, 100, {14, 70, 200, 550, 750, 950}},
217     {"Amber Way", "AMBR", STREET, 200, 3, 100, {16, 80, 220, 600, 800, 1000}},
218     {"Free Parking", "FREE", FREE_PARKING, 0, -1, 0, {}},
219     {"Ruby Rd", "RUBY", STREET, 220, 4, 150, {18, 90, 250, 700, 875, 1050}},
220     {"Chance", "CHNC", CHANCE, 0, -1, 0, {}},
221     {"Garnet Ave", "GRNT", STREET, 220, 4, 150, {18, 90, 250, 700, 875, 1050}},
222     {"Cherry Ln", "CHRY", STREET, 240, 4, 150, {20, 100, 300, 750, 925, 1100}},
223     {"South Rail", "RR3", RAILROAD, 200, 8, 0, {}},
224     {"Gold St", "GOLD", STREET, 260, 5, 150, {22, 110, 330, 800, 975, 1150}},
225     {"Lemon Ave", "LEMN", STREET, 260, 5, 150, {22, 110, 330, 800, 975, 1150}},
226     {"Water Co", "WTR", UTILITY, 150, 9, 0, {}},
227     {"Sunny Way", "SUNY", STREET, 280, 5, 150, {24, 120, 360, 850, 1025, 1200}},
228     {"Go To Jail", "GO\nJAIL", GO_TO_JAIL, 0, -1, 0, {}},
229     {"Fern Ave", "FERN", STREET, 300, 6, 200, {26, 130, 390, 900, 1100, 1275}},
230     {"Ivy Ln", "IVY", STREET, 300, 6, 200, {26, 130, 390, 900, 1100, 1275}},
231     {"Community Chest", "CHST", CHEST, 0, -1, 0, {}},
232     {"Jade Blvd", "JADE", STREET, 320, 6, 200, {28, 150, 450, 1000, 1200, 1400}},
233     {"West Rail", "RR4", RAILROAD, 200, 8, 0, {}},
234     {"Chance", "CHNC", CHANCE, 0, -1, 0, {}},
235     {"Ocean Dr", "OCEN", STREET, 350, 7, 200, {35, 175, 500, 1100, 1300, 1500}},
236     {"Luxury Tax", "LUX", TAX, 100, -1, 0, {}},
237     {"Skyline Pl", "SKY", STREET, 400, 7, 200, {50, 200, 600, 1400, 1700, 2000}},
238 };
239
240 const int BOARD_SIZE = 40, JAIL_INDEX = 10, GO_MONEY = 200, JAIL_FINE = 50, START_MONEY = 1500;
241
242 const SDL_Color GROUP_COLOR[10] = {
243     {140, 85, 55, 255}, {150, 205, 240, 255}, {215, 60, 150, 255}, {245, 145, 30, 255}, {225, 35, 40, 255},
244     {250, 220, 30, 255}, {30, 160, 80, 255}, {30, 80, 190, 255}, {60, 60, 60, 255}, {170, 170, 170, 255}};
245 const SDL_Color PLAYER_COLOR[2] = {{35, 125, 245, 255}, {235, 65, 75, 255}};
246
247 // ----- drawing helpers -----
248 // Tiny 3x5 font: no extra font library or font files needed.
249 const string GLYPHS = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ$+./:;!?,()'"';

```

```

250 const char* FONT[] = {
251     "111101101101111", "010110010010111", "111001111100111", "111001111001111", "101101111001001",
252     "111100111001111", "111100111101111", "111001001001001", "111101111100111", "111101111001111",
253     "01010111101101", "110101110101110", "011100100100011", "110101101101110", "111100110100111",
254     "111100110100100", "011100101101011", "101101111101101", "111010010010111", "001001001101010",
255     "101101110101101", "100100100100111", "10111111101101", "110101101101101", "010101101101010",
256     "110101110100100", "010101101110011", "110101110101101", "011100010001110", "111010010010010",
257     "101101101101111", "101101101101010", "101101111111101", "101101010101101", "101101010010010",
258     "111001010100111", "011110010011110", "000000111000000", "000010111010000", "000010000010000",
259     "001001010100100", "000000000000010", "010010010000010", "111001011000010", "000000000010100",
260     "001010010010001", "100010010010100", "010010000000000", "000111000111000"};
261
262 int textWidth(const string& s, int size) { return s.empty() ? 0 : int(s.size()) * 4 * size - size; }
263
264 void text(SDL_Renderer* r, const string& words, int x, int y, int size) {
265     for (char ch : words) {
266         size_t g = GLYPHS.find(char(toupper(static_cast<unsigned char>(ch))));
267         if (g != string::npos)
268             for (int i = 0; i < 15; i++)
269                 if (FONT[g][i] == '1') {
270                     SDL_Rect pixel = {x + i % 3 * size, y + i / 3 * size, size, size};
271                     SDL_RenderFillRect(r, &pixel);
272                 }
273         x += 4 * size;
274     }
275 }
276
277 void centerText(SDL_Renderer* r, const string& s, int cx, int y, int size) {
278     size_t start = 0;
279     while (true) {
280         size_t end = s.find('\n', start);
281         string line = s.substr(start, end == string::npos ? string::npos : end - start);
282         text(r, line, cx - textWidth(line, size) / 2, y, size);
283         if (end == string::npos) break;
284         start = end + 1;
285         y += 6 * size + 1;
286     }
287 }
288
289 void setColor(SDL_Renderer* r, SDL_Color c) { SDL_SetRenderDrawColor(r, c.r, c.g, c.b, c.a); }
290 void fill(SDL_Renderer* r, int x, int y, int w, int h) {
291     SDL_Rect q = {x, y, w, h};
292     SDL_RenderFillRect(r, &q);
293 }
294 void frame(SDL_Renderer* r, int x, int y, int w, int h, int thick) {
295     for (int i = 0; i < thick; i++) {
296         SDL_Rect q = {x + i, y + i, w - 2 * i, h - 2 * i};
297         SDL_RenderDrawRect(r, &q);
298     }
299 }
300
301 void drawDie(SDL_Renderer* r, int x, int y, int value) {
302     static const char* PIPS[7] = {"000000000", "000010000", "100000001", "100010001",
303     "101000101", "101010101", "101101101"};
304     SDL_SetRenderDrawColor(r, 255, 255, 255, 255);
305     fill(r, x, y, 56, 56);
306     SDL_SetRenderDrawColor(r, 25, 45, 40, 255);
307     frame(r, x, y, 56, 56, 2);
308     for (int i = 0; i < 9; i++)
309         if (PIPS[value][i] == '1') fill(r, x + 9 + (i % 3) * 15, y + 9 + (i / 3) * 15, 9, 9);
310 }
311
312 // Put each space around a circle, clockwise from the top.
313 SDL_Point position(int index, int radius = 330) {
314     double angle = index * 2 * PI / BOARD_SIZE - PI / 2;
315     return {360 + int(radius * cos(angle)), 360 + int(radius * sin(angle))};
316 }
317
318 // ----- game rules -----
319 enum Phase { ROLL, BUY, AUCTION, END, DEBT, OVER };
320 enum Action { A_ROLL, A_PAY_FINE, A_USE_CARD, A_BUY, A_AUCTION, A_BID10, A_BID50, A_BID100, A_DROP,
321     A_END, A_BUILD, A_SELL, A_MORTGAGE, A_UNMORTGAGE, A_BANKRUPT };
322 enum CardKind { MOVE_TO, MONEY, CARD_JAIL, JAIL_FREE, BACK_THREE, REPAIRS, PAY_EACH, COLLECT_EACH };
323
324 struct Card { string text; CardKind kind; int a, b, holder; };
325 struct Deck { vector<Card> cards; vector<int> order; int next = 0; };
326 struct Button { SDL_Rect rect; string label; Action action; };
327
328 string P(int p) { return "P" + to_string(p + 1); }
329 string cash(int m) { return "$" + to_string(m); }
330
331 class Game {
332 public:
333     Board board;

```

```

334 Space* pos[2];
335 int money[2] = {START_MONEY, START_MONEY};
336 bool inJail[2] = {false, false};
337 int jailTries[2] = {0, 0};
338 int creditor[2] = {-1, -1}; // who a player last paid (-1 = bank), used for bankruptcy
339 int turn = 0, turnNumber = 1, doublesCount = 0, die1 = 1, die2 = 1;
340 bool extraRoll = false;
341 Phase phase = ROLL, returnPhase = END;
342 int debtor = 0, winner = -1;
343 int bid = 0, bidder = 0, highBidder = -1, noBidDrops = 0;
344 Space* selected;
345 Deck chance, chest;
346 deque<string> logLines;
347
348 Game() {
349     for (const SpaceInfo& info : BOARD_DATA) board.add(info);
350     pos[0] = pos[1] = selected = board.first();
351     chance.cards = {
352         {"ADVANCE TO GO", MOVE_TO, 0, 0, -1},
353         {"ADVANCE TO SKYLINE PL", MOVE_TO, 39, 0, -1},
354         {"ADVANCE TO RUBY RD", MOVE_TO, 21, 0, -1},
355         {"ADVANCE TO NORTH RAIL", MOVE_TO, 5, 0, -1},
356         {"GO BACK 3 SPACES", BACK_THREE, 0, 0, -1},
357         {"GO TO JAIL", CARD_JAIL, 0, 0, -1},
358         {"GET OUT OF JAIL FREE", JAIL_FREE, 0, 0, -1},
359         {"BANK PAYS YOU $50", MONEY, 50, 0, -1},
360         {"SPEEDING FINE: PAY $15", MONEY, -15, 0, -1},
361         {"REPAIRS: $25 PER HOUSE, $100 PER HOTEL", REPAIRS, 25, 100, -1},
362         {"CHAIRMAN: PAY EACH PLAYER $50", PAY_EACH, 50, 0, -1}};
363     chest.cards = {
364         {"ADVANCE TO GO", MOVE_TO, 0, 0, -1},
365         {"BANK ERROR: COLLECT $200", MONEY, 200, 0, -1},
366         {"DOCTOR FEE: PAY $50", MONEY, -50, 0, -1},
367         {"GO TO JAIL", CARD_JAIL, 0, 0, -1},
368         {"GET OUT OF JAIL FREE", JAIL_FREE, 0, 0, -1},
369         {"TAX REFUND: COLLECT $20", MONEY, 20, 0, -1},
370         {"HOSPITAL: PAY $100", MONEY, -100, 0, -1},
371         {"BIRTHDAY: COLLECT $10 FROM EACH", COLLECT_EACH, 10, 0, -1},
372         {"STREET REPAIRS: $40 PER HOUSE, $115 PER HOTEL", REPAIRS, 40, 115, -1},
373         {"INHERITANCE: COLLECT $100", MONEY, 100, 0, -1}};
374     shuffleDeck(chance);
375     shuffleDeck(chest);
376     log("--- TURN 1: P1 ---", false); // on screen only; the console prints it after the board
377 }
378
379 int actor() const { return phase == DEBT ? debtor : phase == AUCTION ? bidder : turn; }
380
381 // Prints to the console and keeps the last lines for the on-screen log.
382 void log(const string& msg, bool toConsole = true) {
383     if (toConsole) cout << msg << '\n';
384     string m = msg;
385     for (char& c : m) c = char(toupper(static_cast<unsigned char>(c)));
386     while (m.size() > 34) {
387         size_t cut = m.rfind(' ', 34);
388         if (cut == string::npos || cut == 0) cut = 34;
389         logLines.push_back(m.substr(0, cut));
390         m = " " + m.substr(cut + (m[cut] == ' ' ? 1 : 0));
391     }
392     logLines.push_back(m);
393     while (logLines.size() > 20) logLines.pop_front();
394 }
395
396 void shuffleDeck(Deck& d) {
397     d.order.clear();
398     for (int i = 0; i < int(d.cards.size()); i++) d.order.push_back(i);
399     for (int i = int(d.order.size()) - 1; i > 0; i--) swap(d.order[i], d.order[rand() % (i + 1)]);
400     d.next = 0;
401 }
402
403 int cardsHeld(int p) const {
404     int n = 0;
405     for (const Card& c : chance.cards) n += c.holder == p;
406     for (const Card& c : chest.cards) n += c.holder == p;
407     return n;
408 }
409
410 // ----- property queries (each walks the list once: O(n)) -----
411 bool ownsGroup(int p, int g) const {
412     const Space* s = board.first();
413     for (int i = 0; i < board.size(); i++, s = s->next)
414         if (s->group == g && s->owner != p) return false;
415     return true;
416 }
417 int countOwned(int p, int g) const {

```

```

418     int n = 0;
419     const Space* s = board.first();
420     for (int i = 0; i < board.size(); i++, s = s->next) n += s->group == g && s->owner == p;
421     return n;
422 }
423 int groupMinHouses(int g) const {
424     int m = 5;
425     const Space* s = board.first();
426     for (int i = 0; i < board.size(); i++, s = s->next)
427         if (s->group == g && s->houses < m) m = s->houses;
428     return m;
429 }
430 int groupMaxHouses(int g) const {
431     int m = 0;
432     const Space* s = board.first();
433     for (int i = 0; i < board.size(); i++, s = s->next)
434         if (s->group == g && s->houses > m) m = s->houses;
435     return m;
436 }
437 bool groupMortgaged(int g) const {
438     const Space* s = board.first();
439     for (int i = 0; i < board.size(); i++, s = s->next)
440         if (s->group == g && s->mortgaged) return true;
441     return false;
442 }
443 int propsOwned(int p) const {
444     int n = 0;
445     const Space* s = board.first();
446     for (int i = 0; i < board.size(); i++, s = s->next) n += s->owner == p;
447     return n;
448 }
449 int netWorth(int p) const {
450     int total = money[p];
451     const Space* s = board.first();
452     for (int i = 0; i < board.size(); i++, s = s->next)
453         if (s->owner == p) total += (s->mortgaged ? s->cost / 2 : s->cost) + s->houses * s->houseCost;
454     return total;
455 }
456 // Most cash a player could raise by selling buildings and mortgaging.
457 int raisable(int p) const {
458     int total = 0;
459     const Space* s = board.first();
460     for (int i = 0; i < board.size(); i++, s = s->next)
461         if (s->owner == p) total += s->houses * s->houseCost / 2 + (s->mortgaged ? 0 : s->cost / 2);
462     return total;
463 }
464
465 int rentFor(const Space* s) const {
466     if (s->type == STREET) {
467         if (s->houses > 0) return s->rent[s->houses];
468         return ownsGroup(s->owner, s->group) ? s->rent[0] * 2 : s->rent[0]; // full set doubles base rent
469     }
470     int n = countOwned(s->owner, s->group);
471     if (s->type == RAILROAD) return 25 << (n - 1); // 25, 50, 100, 200
472     return (die1 + die2) * (n == 2 ? 10 : 4); // utilities
473 }
474 int unmortgageCost(const Space* s) const { return (s->cost / 2 * 11 + 9) / 10; } // mortgage + 10%
475
476 // ----- what the acting player may do with the selected space -----
477 bool ownedByActor(const Space* s) const { return s && s->owner >= 0 && s->owner == actor(); }
478 bool canBuild(const Space* s) const {
479     return (phase == ROLL || phase == END) && ownedByActor(s) && s->type == STREET &&
480         ownsGroup(s->owner, s->group) && !groupMortgaged(s->group) && s->houses < 5 &&
481         s->houses == groupMinHouses(s->group) && money[s->owner] >= s->houseCost;
482 }
483 bool canSell(const Space* s) const {
484     return (phase == ROLL || phase == BUY || phase == END || phase == DEBT) && ownedByActor(s) &&
485         s->houses > 0 && s->houses == groupMaxHouses(s->group);
486 }
487 bool canMortgage(const Space* s) const {
488     return (phase == ROLL || phase == BUY || phase == END || phase == DEBT) && ownedByActor(s) &&
489         !s->mortgaged && (s->type != STREET || groupMaxHouses(s->group) == 0);
490 }
491 bool canUnmortgage(const Space* s) const {
492     return (phase == ROLL || phase == END) && ownedByActor(s) && s->mortgaged &&
493         money[s->owner] >= unmortgageCost(s);
494 }
495
496 // Buy rule: only an unowned, purchasable property, and only if the player can afford it.
497 bool tryBuy(int p, Space* s) {
498     if (s->type != STREET && s->type != RAILROAD && s->type != UTILITY) {
499         log(P(p) + " CANNOT BUY " + s->name + ": NOT FOR SALE");
500         return false;
501     }

```

```

502     if (s->owner >= 0) {
503         log(P(p) + " CANNOT BUY " + s->name + ": OWNED BY " + P(s->owner));
504         return false;
505     }
506     if (money[p] < s->cost) {
507         log(P(p) + " CANNOT AFFORD " + s->name);
508         return false;
509     }
510     pay(p, -1, s->cost);
511     s->owner = p;
512     log(P(p) + " BOUGHT " + s->name + " FOR " + cash(s->cost));
513     return true;
514 }
515
516 // ----- money and movement -----
517 void pay(int from, int to, int amount) { // -1 means the bank
518     if (from >= 0) {
519         money[from] -= amount;
520         creditor[from] = to;
521     }
522     if (to >= 0) money[to] += amount;
523 }
524
525 void moveForward(int steps) { // 0(steps): one pointer hop per space
526     for (int i = 0; i < steps; i++) {
527         pos[turn] = pos[turn]->next;
528         if (pos[turn] == board.first()) {
529             pay(-1, turn, GO_MONEY);
530             log(P(turn) + " PASSED GO +$200");
531         }
532     }
533 }
534
535 void sendToJail() {
536     pos[turn] = board.at(JAIL_INDEX); // straight to jail: no GO money
537     inJail[turn] = true;
538     jailTries[turn] = 0;
539     doublesCount = 0;
540     extraRoll = false;
541     phase = END;
542     log(P(turn) + " GOES TO JAIL");
543 }
544
545 void landOn() {
546     Space* s = pos[turn];
547     selected = s;
548     phase = END;
549     log(P(turn) + " LANDED ON " + s->name);
550     switch (s->type) {
551     case TAX:
552         pay(turn, -1, s->cost);
553         log(P(turn) + " PAID " + cash(s->cost) + " TAX");
554         break;
555     case GO_TO_JAIL: sendToJail(); break;
556     case CHANCE: drawCard(chance, "CHANCE"); break;
557     case CHEST: drawCard(chest, "CHEST"); break;
558     case JAIL:
559         if (!inJail[turn]) log("JUST VISITING");
560         break;
561     case STREET:
562     case RAILROAD:
563     case UTILITY:
564         if (s->owner < 0) phase = BUY;
565         else if (s->owner != turn && s->mortgaged) log("MORTGAGED, NO RENT");
566         else if (s->owner != turn) {
567             int rent = rentFor(s);
568             pay(turn, s->owner, rent);
569             log(P(turn) + " PAID " + P(s->owner) + " " + cash(rent) + " RENT");
570         }
571         break;
572     default: break;
573     }
574 }
575
576 void drawCard(Deck& d, const string& deckName) {
577     int idx;
578     do { // skip a Get Out of Jail Free card someone is holding
579         idx = d.order[d.next];
580         d.next = (d.next + 1) % int(d.order.size());
581     } while (d.cards[idx].holder >= 0);
582     Card& c = d.cards[idx];
583     log(deckName + ": " + c.text);
584     switch (c.kind) {
585     case MOVE_TO:

```

```

586         moveForward((c.a - pos[turn]->index + BOARD_SIZE) % BOARD_SIZE);
587         landOn();
588         break;
589     case MONEY:
590         if (c.a >= 0) pay(-1, turn, c.a);
591         else pay(turn, -1, -c.a);
592         break;
593     case CARD_JAIL: sendToJail(); break;
594     case JAIL_FREE: c.holder = turn; break;
595     case BACK_THREE: // singly linked: going back 3 = going forward 37, without passing 60
596         for (int i = 0; i < BOARD_SIZE - 3; i++) pos[turn] = pos[turn]->next;
597         landOn();
598         break;
599     case REPAIRS: {
600         int houses = 0, hotels = 0;
601         const Space* s = board.first();
602         for (int i = 0; i < board.size(); i++, s = s->next)
603             if (s->owner == turn) {
604                 if (s->houses == 5) hotels++;
605                 else houses += s->houses;
606             }
607         int cost = houses * c.a + hotels * c.b;
608         pay(turn, -1, cost);
609         log(P(turn) + " PAID " + cash(cost) + " FOR REPAIRS");
610         break;
611     }
612     case PAY_EACH: pay(turn, 1 - turn, c.a); break;
613     case COLLECT_EACH: pay(1 - turn, turn, c.a); break;
614 }
615 }
616
617 void roll() {
618     die1 = rand() % 6 + 1;
619     die2 = rand() % 6 + 1;
620     bool doubles = die1 == die2;
621     log(P(turn) + " ROLLED " + to_string(die1) + " + " + to_string(die2) + (doubles ? " DOUBLES" : ""));
622     if (inJail[turn]) {
623         if (doubles) {
624             inJail[turn] = false;
625             log(P(turn) + " ROLLED OUT OF JAIL");
626         } else if (++jailTries[turn] == 3) {
627             pay(turn, -1, JAIL_FINE);
628             inJail[turn] = false;
629             log("3RD MISS: " + P(turn) + " PAYS $50 AND MOVES");
630         } else {
631             log("NO DOUBLES, STAYS IN JAIL (" + to_string(jailTries[turn]) + "/3)");
632             extraRoll = false;
633             phase = END;
634             return;
635         }
636     }
637     moveForward(die1 + die2);
638     landOn();
639     extraRoll = false; // leaving jail on doubles does not give another roll
640     return;
641 }
642 if (doubles && ++doublesCount == 3) {
643     log("3 DOUBLES IN A ROW!");
644     sendToJail();
645     return;
646 }
647 moveForward(die1 + die2);
648 landOn();
649 extraRoll = doubles && !inJail[turn];
650 }
651
652 void printStandings() const {
653     cout << " [after turn " << turnNumber << " ] ";
654     for (int p = 0; p < 2; p++)
655         cout << P(p) << " $" << money[p] << ", " << propsOwned(p) << " properties, net worth $"
656             << netWorth(p) << (inJail[p] ? " (in jail)" : "") << (p == 0 ? " | " : "\n");
657 }
658
659 void endTurn() {
660     printStandings();
661     turn = 1 - turn;
662     turnNumber++;
663     doublesCount = 0;
664     extraRoll = false;
665     phase = ROLL;
666     selected = pos[turn];
667     log("--- TURN " + to_string(turnNumber) + ": " + P(turn) + " ---");
668 }
669
670 void bankrupt(int p) {

```

```

670     int to = creditor[p];
671     log(P(p) + " IS BANKRUPT!");
672     int buildingCash = 0;
673     Space* s = board.first();
674     for (int i = 0; i < board.size(); i++, s = s->next)
675     {
676         if (s->owner == p) {
677             buildingCash += s->houses * s->houseCost / 2; // buildings go back to the bank at half price
678             s->houses = 0;
679             s->owner = to; // creditor (or bank) takes the deeds
680             if (to < 0) s->mortgaged = false;
681         }
682         if (to >= 0) money[to] += money[p] + buildingCash; // creditor only gets what the debtor really had
683         money[p] = 0;
684         winner = 1 - p;
685         phase = OVER;
686         log(P(winner) + " WINS THE GAME!");
687         cout << "\n=== GAME OVER ===\n";
688         printStandings();
689         cout << "Final board:\n";
690         board.print();
691     }
692     // A player below $0 must sell/mortgage; if even that can't cover it, they're out.
693     void checkDebt() {
694         if (phase == OVER) return;
695         for (int p : {turn, 1 - turn}) {
696             if (money[p] >= 0) continue;
697             if (money[p] + raisable(p) < 0) {
698                 bankrupt(p);
699                 return;
700             }
701             if (phase != DEBT) {
702                 returnPhase = phase;
703                 log(P(p) + " OWES " + cash(-money[p]) + ": SELL OR MORTGAGE");
704             }
705             phase = DEBT;
706             debtor = p;
707             return;
708         }
709         if (phase == DEBT) {
710             phase = returnPhase;
711             log("DEBT PAID OFF");
712         }
713     }
714
715     void doAction(Action a) {
716         if (phase == OVER) return;
717         int me = actor();
718         Space* here = pos[turn];
719         Space* s = selected;
720         switch (a) {
721             case A_ROLL: roll(); break;
722             case A_PAY_FINE:
723                 pay(turn, -1, JAIL_FINE);
724                 inJail[turn] = false;
725                 log(P(turn) + " PAID $50 TO LEAVE JAIL");
726                 break;
727             case A_USE_CARD: {
728                 bool used = false;
729                 for (Deck* d : {&chance, &chest})
730                     for (Card& c : d->cards)
731                         if (!used && c.holder == turn) {
732                             c.holder = -1;
733                             used = true;
734                         }
735                 inJail[turn] = false;
736                 log(P(turn) + " USED GET OUT OF JAIL FREE");
737                 break;
738             }
739             case A_BUY:
740                 if (tryBuy(turn, here)) phase = END;
741                 break;
742             case A_AUCTION:
743                 log(P(turn) + " PASSED. AUCTION FOR " + here->name);
744                 phase = AUCTION;
745                 bid = 0;
746                 highBidder = -1;
747                 bidder = 1 - turn;
748                 noBidDrops = 0;
749                 break;
750             case A_BID10:
751             case A_BID50:
752             case A_BID100:
753                 bid += a == A_BID10 ? 10 : a == A_BID50 ? 50 : 100;

```

```

754         highBidder = bidder;
755         log(P(bidder) + " BIDS " + cash(bid));
756         bidder = 1 - bidder;
757         break;
758     case A_DROP:
759         log(P(bidder) + " DROPS OUT");
760         if (highBidder >= 0) {
761             pay(highBidder, -1, bid);
762             here->owner = highBidder;
763             phase = END;
764             log(P(highBidder) + " WINS " + here->name + " FOR " + cash(bid));
765         } else if (++noBidDrops == 2) {
766             log("NO BIDS. BANK KEEPS IT");
767             phase = END;
768         } else bidder = 1 - bidder;
769         break;
770     case A_END: endTurn(); break;
771     case A_BUILD:
772         pay(me, -1, s->houseCost);
773         s->houses++;
774         log(P(me) + (s->houses == 5 ? " BUILT A HOTEL ON " : " BUILT A HOUSE ON ") + s->name);
775         break;
776     case A_SELL:
777         log(P(me) + (s->houses == 5 ? " SOLD A HOTEL ON " : " SOLD A HOUSE ON ") + s->name);
778         s->houses--;
779         pay(-1, me, s->houseCost / 2);
780         break;
781     case A_MORTGAGE:
782         s->mortgaged = true;
783         pay(-1, me, s->cost / 2);
784         log(P(me) + " MORTGAGED " + s->name + " " + cash(s->cost / 2));
785         break;
786     case A_UNMORTGAGE:
787         pay(me, -1, unmortgageCost(s));
788         s->mortgaged = false;
789         log(P(me) + " UNMORTGAGED " + s->name);
790         break;
791     case A_BANKRUPT: bankrupt(me); return;
792 }
793 checkDebt();
794 }
795
796 vector<Button> buttons() const {
797     vector<Button> list;
798     auto add = [&](const string& label, Action a) {
799         int i = int(list.size());
800         list.push_back({736 + (i % 2) * 146, 268 + (i / 2) * 38, 138, 30}, label, a);
801     };
802     switch (phase) {
803     case ROLL:
804         if (inJail[turn]) {
805             add("ROLL DOUBLES", A_ROLL);
806             if (money[turn] >= JAIL_FINE) add("PAY $50", A_PAY_FINE);
807             if (cardsHeld(turn) > 0) add("USE CARD", A_USE_CARD);
808         } else add("ROLL DICE", A_ROLL);
809         break;
810     case BUY:
811         if (money[turn] >= pos[turn]->cost) add("BUY " + cash(pos[turn]->cost), A_BUY);
812         add("AUCTION", A_AUCTION);
813         break;
814     case AUCTION:
815         if (money[bidder] >= bid + 10) add("BID +$10", A_BID10);
816         if (money[bidder] >= bid + 50) add("BID +$50", A_BID50);
817         if (money[bidder] >= bid + 100) add("BID +$100", A_BID100);
818         add("DROP OUT", A_DROP);
819         break;
820     case END: add(extraRoll ? "ROLL AGAIN" : "END TURN", extraRoll ? A_ROLL : A_END); break;
821     default: break;
822     }
823     const Space* s = selected;
824     if (canBuild(s)) add(s->houses == 4 ? "BUILD HOTEL" : "BUILD HOUSE", A_BUILD);
825     if (canSell(s)) add(s->houses == 5 ? "SELL HOTEL" : "SELL HOUSE", A_SELL);
826     if (canMortgage(s)) add("MORTGAGE " + cash(s->cost / 2), A_MORTGAGE);
827     if (canUnmortgage(s)) add("UNMORT " + cash(unmortgageCost(s)), A_UNMORTGAGE);
828     if (phase == DEBT) add("BANKRUPT", A_BANKRUPT);
829     return list;
830 }
831
832 void primaryAction() { // SPACE key
833     if (phase == DEBT || phase == OVER) return;
834     vector<Button> b = buttons();
835     if (!b.empty()) doAction(b[0].action);
836 }
837

```

```

838 // A simple computer player, handy for simulating turns quickly.
839 void autoStep() {
840     switch (phase) {
841         case ROLL: doAction(inJail[turn] && cardsHeld(turn) > 0 ? A_USE_CARD : A_ROLL); break;
842         case BUY: doAction(money[turn] >= pos[turn]->cost ? A_BUY : A_AUCTION); break;
843         case AUCTION:
844             doAction(bid + 10 <= pos[turn]->cost && money[bidder] - bid - 10 >= 100 ? A_BID10 : A_DROP);
845             break;
846         case END: {
847             bool built = true;
848             while (built && phase == END) {
849                 built = false;
850                 Space* s = board.first();
851                 for (int i = 0; i < board.size(); i++, s = s->next) {
852                     selected = s;
853                     if (canBuild(s) && money[turn] - s->houseCost >= 150) {
854                         doAction(A_BUILD);
855                         built = true;
856                     }
857                 }
858             }
859             selected = pos[turn];
860             doAction(extraRoll ? A_ROLL : A_END);
861             break;
862         }
863         case DEBT:
864             while (phase == DEBT) { // sell buildings first, then mortgage
865                 Space* pick = nullptr;
866                 Action act = A_BANKRUPT;
867                 Space* s = board.first();
868                 for (int i = 0; i < board.size() && !pick; i++, s = s->next)
869                     if (canSell(s)) pick = s, act = A_SELL;
870                 s = board.first();
871                 for (int i = 0; i < board.size() && !pick; i++, s = s->next)
872                     if (canMortgage(s)) pick = s, act = A_MORTGAGE;
873                 if (pick) selected = pick;
874                 doAction(act);
875             }
876             break;
877         case OVER: break;
878     }
879 }
880
881 void clickBoard(int x, int y) {
882     Space* s = board.first();
883     for (int i = 0; i < board.size(); i++, s = s->next) {
884         SDL_Point p = position(s->index);
885         if (abs(x - p.x) <= 17 && abs(y - p.y) <= 17) {
886             selected = s;
887             return;
888         }
889     }
890 }
891
892 string phaseText() const {
893     switch (phase) {
894         case ROLL:
895             return inJail[turn] ? P(turn) + " IN JAIL: TRY " + to_string(jailTries[turn] + 1) + "/" + 3
896                 : P(turn) + ": ROLL THE DICE";
897         case BUY: return P(turn) + ": BUY IT OR AUCTION IT";
898         case AUCTION:
899             return "BID " + cash(bid) + (highBidder >= 0 ? " BY " + P(highBidder) : "") + ", " + P(bidder) +
900                 " TO ACT";
901         case END: return extraRoll ? "DOUBLES! " + P(turn) + " ROLLS AGAIN" : P(turn) + ": BUILD OR END TURN";
902         case DEBT: return P(debtor) + " MUST RAISE " + cash(-money[debtor]);
903         case OVER: return "GAME OVER: " + P(winner) + " WINS";
904     }
905     return "";
906 }
907
908 vector<string> infoLines(const Space* s) const {
909     vector<string> out;
910     string owner = s->owner < 0 ? "BANK" : P(s->owner);
911     string ownerLine = "OWNER " + owner + (s->mortgaged ? " MORTGAGED" : "");
912     switch (s->type) {
913         case STREET:
914             out.push_back("PRICE " + cash(s->cost) + " HOUSE " + cash(s->houseCost));
915             out.push_back(ownerLine);
916             out.push_back("RENT " + cash(s->rent[0]) + " FULL SET " + cash(s->rent[0] * 2));
917             out.push_back("1H " + cash(s->rent[1]) + " 2H " + cash(s->rent[2]) + " 3H " + cash(s->rent[3]));
918             out.push_back("4H " + cash(s->rent[4]) + " HOTEL " + cash(s->rent[5]));
919             out.push_back(s->houses == 5 ? "BUILT: HOTEL" : "BUILT: " + to_string(s->houses) + " HOUSES");
920             break;
921         case RAILROAD:

```

```

922         out.push_back("PRICE " + cash(s->cost));
923         out.push_back(ownerLine);
924         out.push_back("RENT $25 / 50 / 100 / 200");
925         out.push_back("BY NUMBER OF RAILROADS OWNED");
926         break;
927     case UTILITY:
928         out.push_back("PRICE " + cash(s->cost));
929         out.push_back(ownerLine);
930         out.push_back("RENT 4X DICE, 10X IF BOTH OWNED");
931         break;
932     case TAX: out.push_back("PAY " + cash(s->cost) + " TO THE BANK"); break;
933     case GO_SPACE: out.push_back("COLLECT $200 EVERY TIME"); out.push_back("YOU PASS OR LAND ON GO"); break;
934     case JAIL:
935         out.push_back("JUST VISITING IF YOU LAND HERE");
936         out.push_back("GET OUT: ROLL DOUBLES, PAY $50,");
937         out.push_back("OR USE A GET OUT OF JAIL CARD");
938         out.push_back("MUST PAY $50 AFTER 3 MISSES");
939         break;
940     case GO_TO_JAIL: out.push_back("GO DIRECTLY TO JAIL"); out.push_back("DO NOT PASS GO, NO $200"); break;
941     case FREE_PARKING: out.push_back("NOTHING HAPPENS HERE"); break;
942     case CHANCE:
943     case CHEST: out.push_back("DRAW A CARD AND FOLLOW IT"); break;
944     }
945     if (s->owner >= 0 && !s->mortgaged && s->type != UTILITY) out.push_back("RENT NOW " + cash(rentFor(s)));
946     return out;
947 }
948
949 void drawTile(SDL_Renderer* r, const Space* s) const {
950     SDL_Point p = position(s->index);
951     int x = p.x - 17, y = p.y - 17;
952     if (s->mortgaged) SDL_SetRenderDrawColor(r, 150, 150, 140, 255);
953     else SDL_SetRenderDrawColor(r, 239, 237, 219, 255);
954     fill(r, x, y, 34, 34);
955     if (s->group >= 0) {
956         setColor(r, GROUP_COLOR[s->group]);
957         fill(r, x, y, 34, 6);
958     }
959     SDL_SetRenderDrawColor(r, 25, 45, 40, 255);
960     bool twoLines = s->label.find('\n') != string::npos;
961     centerText(r, s->label, p.x, twoLines ? p.y - 12 : p.y - 5, 2);
962     if (s->houses == 5) {
963         SDL_SetRenderDrawColor(r, 210, 30, 40, 255);
964         fill(r, p.x - 7, y + 26, 14, 5);
965     } else {
966         SDL_SetRenderDrawColor(r, 30, 150, 60, 255);
967         for (int i = 0; i < s->houses; i++) fill(r, x + 4 + i * 7, y + 26, 5, 5);
968     }
969     if (s->owner >= 0) {
970         setColor(r, PLAYER_COLOR[s->owner]);
971         frame(r, x, y, 34, 34, 2);
972     }
973     if (s == selected) {
974         SDL_SetRenderDrawColor(r, 255, 220, 60, 255);
975         frame(r, x - 4, y - 4, 42, 42, 2);
976     }
977 }
978
979 void draw(SDL_Renderer* r) const {
980     SDL_SetRenderDrawColor(r, 22, 65, 56, 255);
981     SDL_RenderClear(r);
982
983     // Circular path behind the spaces.
984     SDL_SetRenderDrawColor(r, 110, 150, 130, 255);
985     for (int i = 0; i < BOARD_SIZE; i++) {
986         SDL_Point a = position(i), b = position((i + 1) % BOARD_SIZE);
987         SDL_RenderDrawLine(r, a.x, a.y, b.x, b.y);
988     }
989
990     // Walk the linked list to draw the board.
991     const Space* s = board.first();
992     for (int i = 0; i < board.size(); i++, s = s->next) drawTile(r, s);
993
994     // Player tokens (jailed players get bars).
995     for (int j = 0; j < 2; j++) {
996         if (phase == OVER && j != winner) continue;
997         SDL_Point p = position(pos[j]->index, 290);
998         int x = p.x - 14 + j * 16, y = p.y - 6;
999         setColor(r, PLAYER_COLOR[j]);
1000         fill(r, x, y, 12, 12);
1001         SDL_SetRenderDrawColor(r, 255, 255, 255, 255);
1002         frame(r, x, y, 12, 12, 1);
1003         if (inJail[j]) {
1004             SDL_SetRenderDrawColor(r, 25, 45, 40, 255);
1005             fill(r, x + 3, y, 2, 12);

```

```

1006         fill(r, x + 7, y, 2, 12);
1007     }
1008 }
1009
1010 // Center: turn, whose move, dice, hints.
1011 SDL_SetRenderDrawColor(r, 170, 205, 185, 255);
1012 centerText(r, "TURN " + to_string(turnNumber), 360, 196, 2);
1013 if (phase == OVER) {
1014     setColor(r, PLAYER_COLOR[winner]);
1015     centerText(r, P(winner) + " WINS!", 360, 222, 6);
1016 } else {
1017     setColor(r, PLAYER_COLOR[actor()]);
1018     string who = phase == DEBT ? " IN DEBT" : phase == AUCTION ? " TO BID" : " TO PLAY";
1019     centerText(r, P(actor()) + who, 360, 226, 4);
1020 }
1021 drawDie(r, 296, 326, die1);
1022 drawDie(r, 368, 326, die2);
1023 SDL_SetRenderDrawColor(r, 170, 205, 185, 255);
1024 centerText(r, "SPACE: MAIN ACTION", 360, 410, 2);
1025 centerText(r, "A: AUTO-PLAY ONE STEP", 360, 426, 2);
1026 centerText(r, "CLICK A SPACE TO INSPECT", 360, 442, 2);
1027
1028 // Side panel.
1029 SDL_SetRenderDrawColor(r, 14, 42, 37, 255);
1030 fill(r, 720, 0, 320, 720);
1031 for (int p = 0; p < 2; p++) {
1032     int y = 14 + p * 56;
1033     if (p == turn && phase != OVER) {
1034         SDL_SetRenderDrawColor(r, 255, 220, 60, 255);
1035         fill(r, 726, y, 4, 34);
1036     }
1037     setColor(r, PLAYER_COLOR[p]);
1038     text(r, P(p) + " " + cash(money[p]), 736, y, 3);
1039     SDL_SetRenderDrawColor(r, 200, 215, 205, 255);
1040     string status = to_string(propsOwned(p)) + " PROPS WORTH " + cash(netWorth(p));
1041     if (inJail[p]) status += " JAIL";
1042     if (cardsHeld(p) > 0) status += " CARD";
1043     text(r, status, 736, y + 22, 2);
1044 }
1045 SDL_SetRenderDrawColor(r, 60, 100, 88, 255);
1046 fill(r, 730, 126, 300, 1);
1047
1048 if (selected) {
1049     if (selected->group >= 0) {
1050         setColor(r, GROUP_COLOR[selected->group]);
1051         fill(r, 736, 138, 10, 10);
1052     }
1053     SDL_SetRenderDrawColor(r, 255, 255, 255, 255);
1054     text(r, selected->name, 752, 138, 2);
1055     SDL_SetRenderDrawColor(r, 200, 215, 205, 255);
1056     int y = 158;
1057     for (const string& line : infoLines(selected)) {
1058         text(r, line, 736, y, 2);
1059         y += 14;
1060     }
1061 }
1062
1063 for (const Button& b : buttons()) {
1064     SDL_SetRenderDrawColor(r, 236, 233, 210, 255);
1065     SDL_RenderFillRect(r, &b.rect);
1066     SDL_SetRenderDrawColor(r, 25, 45, 40, 255);
1067     centerText(r, b.label, b.rect.x + b.rect.w / 2, b.rect.y + 10, 2);
1068 }
1069
1070 SDL_SetRenderDrawColor(r, 255, 220, 60, 255);
1071 text(r, phaseText(), 736, 392, 2);
1072 SDL_SetRenderDrawColor(r, 60, 100, 88, 255);
1073 fill(r, 730, 410, 300, 1);
1074 SDL_SetRenderDrawColor(r, 200, 215, 205, 255);
1075 int y = 420;
1076 for (const string& line : logLines) {
1077     text(r, line, 736, y, 2);
1078     y += 14;
1079 }
1080 }
1081 };
1082
1083 // Console simulation: the computer plays both sides for a set number of turns and prints every result.
1084 void simulateTurns(int turns) {
1085     cout << "\n=== PART 2: MONOPOLY SIMULATION (" << turns << " TURNS) ===\n";
1086     Game sim;
1087     cout << "Board (circular linked list):\n";
1088     sim.board.print();
1089     cout << "\n--- TURN 1: P1 ---\n";

```

```

1090 while (sim.phase != OVER) {
1091     if (sim.turnNumber == turns && sim.phase == END && !sim.extraRoll) { // stop after the last turn
1092         sim.printStandings();
1093         break;
1094     }
1095     sim.autoStep();
1096 }
1097
1098 cout << "\n--- Buy rule check ---\n";
1099 Space* s = sim.board.first();
1100 Space* owned = nullptr;
1101 for (int i = 0; i < sim.board.size() && !owned; i++, s = s->next)
1102     if (s->owner >= 0) owned = s;
1103 if (owned) {
1104     int other = 1 - owned->owner;
1105     check(!sim.tryBuy(other, owned), "player cannot buy a property someone already owns");
1106 }
1107 check(!sim.tryBuy(0, sim.board.first()), "player cannot buy G0 (not a property)");
1108
1109 cout << "\n--- Results after " << turns << " turns ---\n";
1110 for (int p = 0; p < 2; p++)
1111     cout << " " << P(p) << ": $" << sim.money[p] << " cash, " << sim.propsOwned(p)
1112         << " properties, net worth $" << sim.netWorth(p) << (sim.inJail[p] ? ", in jail" : "")
1113         << ", on " << sim.pos[p]->name << '\n';
1114 int leader = sim.phase == OVER ? sim.winner : (sim.netWorth(0) >= sim.netWorth(1) ? 0 : 1);
1115 cout << " " << (sim.phase == OVER ? "Winner: " : "Leader: ") << P(leader) << '\n';
1116 cout << "Final board:\n";
1117 sim.board.print();
1118 }
1119
1120 int main() {
1121     srand(static_cast<unsigned>(time(nullptr)));
1122     testLinkedList();
1123     simulateTurns(12);
1124
1125     cout << "\n=== PART 2: MONOPOLY GUI (new game) ===\n--- TURN 1: P1 ---\n";
1126     if (SDL_Init(SDL_INIT_VIDEO) != 0) {
1127         cerr << SDL_GetError() << '\n';
1128         return 1;
1129     }
1130     SDL_Window* window = SDL_CreateWindow("Monopoly | SPACE: action | A: auto | click spaces to inspect",
1131                                           SDL_WINDOWPOS_CENTERED, SDL_WINDOWPOS_CENTERED, 1040, 720, 0);
1132     SDL_Renderer* r = window ? SDL_CreateRenderer(window, -1, SDL_RENDERER_SOFTWARE) : nullptr;
1133     if (!r) {
1134         cerr << SDL_GetError() << '\n';
1135         if (window) SDL_DestroyWindow(window);
1136         SDL_Quit();
1137         return 1;
1138     }
1139
1140     Game game;
1141
1142     SDL_Rect diceArea = {296, 326, 128, 56};
1143     bool running = true;
1144     SDL_Event event;
1145     while (running) {
1146         while (SDL_PollEvent(&event)) {
1147             if (event.type == SDL_QUIT) running = false;
1148             if (event.type == SDL_KEYDOWN) {
1149                 if (event.key.keysym.sym == SDLK_SPACE && !event.key.repeat) game.primaryAction();
1150                 if (event.key.keysym.sym == SDLK_a) game.autoStep(); // hold A to fast-forward
1151             }
1152             if (event.type == SDL_MOUSEBUTTONDOWN && event.button.button == SDL_BUTTON_LEFT) {
1153                 SDL_Point mouse = {event.button.x, event.button.y};
1154                 bool handled = false;
1155                 for (const Button& b : game.buttons())
1156                     if (SDL_PointInRect(&mouse, &b.rect)) {
1157                         game.doAction(b.action);
1158                         handled = true;
1159                         break;
1160                     }
1161                 if (!handled && SDL_PointInRect(&mouse, &diceArea)) {
1162                     if (game.phase == ROLL || (game.phase == END && game.extraRoll)) game.doAction(A_ROLL);
1163                     handled = true;
1164                 }
1165                 if (!handled) game.clickBoard(mouse.x, mouse.y);
1166             }
1167         }
1168         game.draw(r);
1169         SDL_RenderPresent(r);
1170         SDL_Delay(16);
1171     }
1172
1173     if (game.phase != OVER) {

```

```

1174         cout << "\n=== WINDOW CLOSED: FINAL RESULTS ===\n";
1175         game.printStandings();
1176         cout << "Final board:\n";
1177         game.board.print();
1178     }
1179     SDL_DestroyRenderer(r);
1180     SDL_DestroyWindow(window);
1181     SDL_Quit();
1182     // The Board destructor deletes each node exactly once (it stops after count nodes).
1183 }

```